

Course 1: Problem Solving through Programming in Java

CO No.	Course Outcome Statement (Elaborated)	Mapped BTL Level
CO1	<i>Apply fundamental programming constructs such as data types, operators, conditional and iterative statements in Java to develop logic-based solutions for basic computational problems.</i> Students will learn to design simple algorithms, trace execution, and validate logic through hands-on coding tasks.	BTL-3
CO2	<i>Design, trace, and optimize algorithms using one-dimensional and two-dimensional arrays to solve mathematical, quantitative, and real-world problems efficiently through search, sort, and matrix manipulation techniques.</i> Students will also analyze the efficiency and accuracy of algorithmic approaches.	BTL-4
CO3	<i>Construct and evaluate advanced problem-solving logic using strings, recursion, and bitwise operations for solving complex mathematical, pattern-based, and combinatorial problems relevant to competitive coding platforms.</i> Students will be able to integrate mathematical reasoning and pattern recognition into coding strategies.	BTL-4
CO4	<i>Develop structured and modular programs by applying object-oriented programming principles such as encapsulation, abstraction, and modularization using Java classes, methods, and constructors.</i> Students will transition from procedural to modular design thinking.	BTL-3
CO5	<i>Design extensible and reusable Java programs employing inheritance, polymorphism, abstract classes, interfaces, and reflection API to solve domain-oriented problems with clarity and maintainability.</i> Students will model real-world entities and relationships through effective OOP architecture.	BTL-5
CO6	<i>Implement robust, scalable, and generic Java applications integrating exception handling, file I/O, generics, and collections framework, along with functional programming constructs to handle real-world data-driven tasks.</i> Students will demonstrate ability to write production-level, fault-tolerant programs.	BTL-4

#	Module Title & Focus	Detailed Syllabus / Topics to be Covered
1	Foundations of Logic Building & Control Structures	• Problem-solving methodology and flowchart design • Introduction to Java programming model (syntax, variables, data types, type casting, operators) • Input/output operations (Scanner, BufferedReader) • Control flow: if-else, nested if, switch-case • Iterative constructs: for, while, do-while • Logical reasoning and flow tracing through dry runs • Pattern printing and arithmetic-based problem logic • Debugging and error tracing techniques
2	1D, 2D Arrays and Optimized Algorithm Design	• Concept of arrays and memory representation • 1D array operations: creation, traversal, insertion, deletion, rotation, merging • 2D arrays: matrix representation and manipulation • Searching techniques – Linear & Binary search with complexity analysis • Sorting techniques – Bubble, Selection, Insertion, Merge, Quick Sort • Optimization strategies: two-pointer technique, prefix sum, sliding window • Matrix algorithms – transpose, rotation, diagonal operations • CodeChef-style problem solving using arrays and loops
3	Strings, Bitwise Operations, and Recursion	• String handling and immutability • String vs StringBuilder vs StringBuffer • Common string problems: palindrome, anagram, substring, frequency count, character manipulation • Regular Expressions (regex) – pattern matching and text validation • Bitwise operators (AND, OR, XOR, NOT, shifts, masks) • Bit manipulation tricks for optimization (checking even/odd, power of 2, swapping, subset generation) • Recursion fundamentals, base & recursive cases, tracing stack frames • Recursive problem-solving: factorial, Fibonacci, backtracking (n-Queens, subset sum) • Quantitative and mathematics-based logical problems
4	Object-Oriented Programming – Classes, Objects, and Methods	• Transition from procedural to object-oriented logic • Defining and using classes & objects • Methods, parameters, return types • Method overloading, constructors, and use of this keyword • Access specifiers and encapsulation • Static data and methods • Design of simple OOP-based applications (banking system, student result system, etc.) • Modularization of logic through classes
5	Inheritance, Polymorphism, Abstraction & Interfaces	• Concept of inheritance and types • Method overriding, super, and final keyword • Abstract classes and abstract methods • Interfaces and multiple inheritance in Java • Polymorphism (compile-time and runtime) • Dynamic binding and late method resolution • Reflection API – introspecting class members at runtime • OOP mini-project integrating multiple classes • Intro to design patterns – factory, strategy, and template
6	Exception Handling, File I/O, Generics & Collections Framework	• Types of exceptions, hierarchy, try-catch-finally, throw and throws • Custom exception classes • File handling: byte stream and character stream • Reading/writing files using FileInputStream, FileOutputStream, FileReader, FileWriter, and buffered streams • Serialization and Deserialization • Generics – parameterized classes and methods • Java Collections Framework – List, Set, Map, Queue and their implementations (ArrayList, HashSet, HashMap, PriorityQueue, etc.) • Functional Programming in Java – Lambda expressions, Stream API • Capstone mini-project integrating file I/O, collections, and exception handling